

Hearable devices with sound bubbles

Received: 19 December 2023

Accepted: 2 October 2024

Published online: 14 November 2024



Tuochao Chen^{1,4}✉, Malek Itani^{1,4}✉, Sefik Emre Eskimez²,
Takuya Yoshioka³ & Shyamnath Gollakota¹✉

The human auditory system has a limited ability to perceive distance and distinguish speakers in crowded settings. A headset technology that can create a sound bubble in which all speakers within the bubble are audible but speakers and noise outside the bubble are suppressed could augment human hearing. However, developing such technology is challenging. Here, we report an intelligent headset system capable of creating sound bubbles. The system is based on real-time neural networks that use acoustic data from up to six microphones integrated into noise-cancelling headsets and are run on the device, processing 8 ms audio chunks in 6.36 ms on an embedded central processing unit. Our neural networks can generate sound bubbles with programmable radii between 1 m and 2 m, and with output signals that reduce the intensity of sounds outside the bubble by 49 dB. With previously unseen environments and wearers, our system can focus on up to two speakers within the bubble, with one to two interfering speakers and noise outside the bubble.

Noise-cancelling headphones can suppress sounds around a wearer, but cannot perceive distance or selectively program acoustic scenes based on speaker distances. The distance perception of the human auditory system is also limited¹, and although we can determine the angular direction of a sound source, estimating distance is more challenging². Distance perception becomes even more challenging with unfamiliar sounds in unknown environments². As interference and noise become louder, our ability to focus on nearby speakers becomes more difficult^{3,4}.

The development of headset technology that can create a sound bubble in which all speakers within the bubble are audible, but speakers and noise outside the bubble are suppressed, is challenging. Such a system could though be of use in a variety of scenarios. For example, it could be used by someone who wishes to eliminate noise in a restaurant and tune-in to a conversation at their table. Alternatively, it could be used in a conference room with simultaneous conversations, where a person can exclusively hear the discussion within their bubble (Fig. 1a). Furthermore, it could be used in noisy environments such as airplanes, tuning out distant speakers and ambient noises, but retaining the ability to hear a passing flight attendant who enters the wearer's bubble. Such scenarios, however, require noise-cancelling

headsets that can suppress all sounds and also play back sounds within the bubble.

Creating intelligent headsets capable of creating sound bubbles requires addressing challenges related to hearable systems and real-time deep-learning algorithms. First, the system needs to identify all speakers within the bubble based on their distance from the wearer and separate the speakers within the bubble from those outside. This must work when the speakers outside the bubble are louder and therefore cannot use only the amplitude information. Second, the sound output should be synchronized with a user's visual senses. This requires neural networks that run on the device and in real time, using only limited computing capabilities and satisfying latency requirements of less than 20–30 ms (refs. 5,6). Third, the neural networks should have the ability to create different bubble sizes and must support configurable distance thresholds. Finally, the neural network must generalize to the real world, unseen users and reverberant environments. This is challenging as reverberations in real-world environments can vary across rooms. Additionally, the signals received at the microphones experience reflections from the human head (head-related transfer functions (HRTFs)). These reflections can vary across users and, in turn, affect the distance estimates.

¹Paul G. Allen School of Computer Science and Engineering, University of Washington, Seattle, WA, USA. ²Microsoft, Seattle, WA, USA. ³AssemblyAI, San Francisco, CA, USA. ⁴These authors contributed equally: Tuochao Chen, Malek Itani. ✉e-mail: tuochao@cs.washington.edu; malek@cs.washington.edu; gshyam@cs.washington.edu

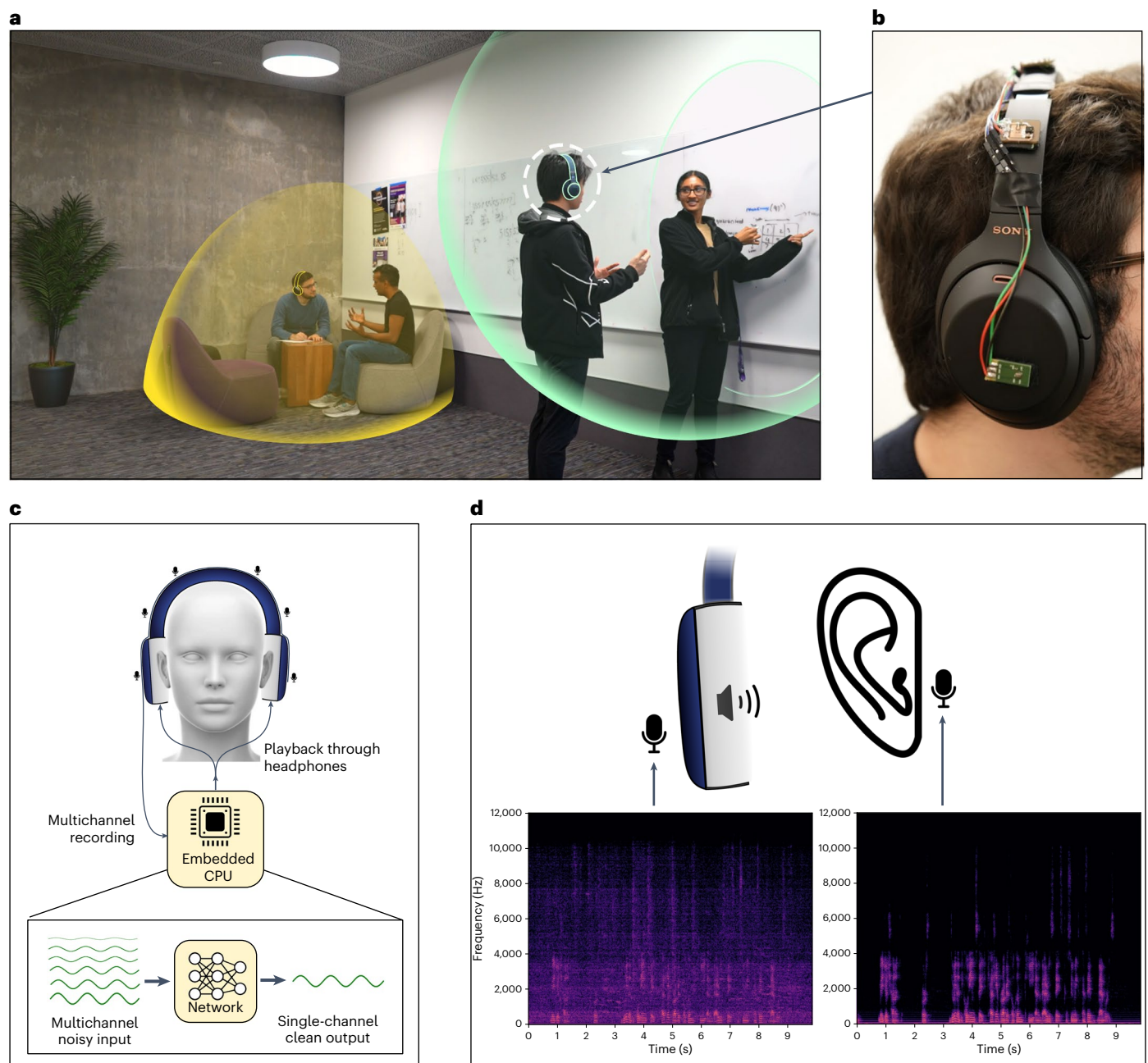


Fig. 1 | Creating sound bubbles using hearables. **a**, Participants using our prototype headset to create distinct programmable sound bubbles to listen to the speakers around them. The wearers hear the sounds from inside their respective bubbles, whereas sounds originating from outside these bubbles are suppressed. **b**, Side view of our system, which consists of an array of microphones attached along the headband and earcups of a pair of noise-cancelling

headphones. **c**, High-level overview of our proposed system. Sound is captured via a six-channel microphone array, processed on an embedded CPU to remove sounds outside the bubble and finally played back into the noise-cancelling headphones. **d**, Real-world spectrogram representations of an audio signal, one as it is captured by a microphone on the exterior of the device and the other as it is captured by a microphone inside an earcup.

In this Article, we report a system that can be used to create programmable sound bubbles on headsets using real-time neural networks that run on embedded central processing units (CPUs). The networks can create sound bubbles with radii of 1 m, 1.5 m and 2 m. Using an Orange Pi 5B, our model can process 8 ms audio chunks at 24 kHz in an average inference time of 7.30 ms with a standard deviation of 0.11 ms. Using a Raspberry Pi 4B, the audio chunks can be processed in an average inference time of 6.36 ms with a standard deviation of 0.18 ms.

Generalization to practical wearers requires a dataset that captures how distance and HRTFs interact with real-world acoustic environments. We, thus, build a robotic platform capable of automating this

data collection using headsets. This provides a dataset of end-to-end audio recordings that captures both head-related reflections and reverberations as a function of distance in real-world environments, with mannequins as well as humans, spanning a total of 15.85 h in 22 different acoustic indoor environments (including offices, living spaces, conference rooms, classrooms and laboratories).

We develop a training methodology that allows our model to achieve real-world generalization to previously unseen acoustic environments and wearers. We incorporate various data augmentation techniques to account for diverse human anthropometric measurements, environmental noise and small changes in the microphone

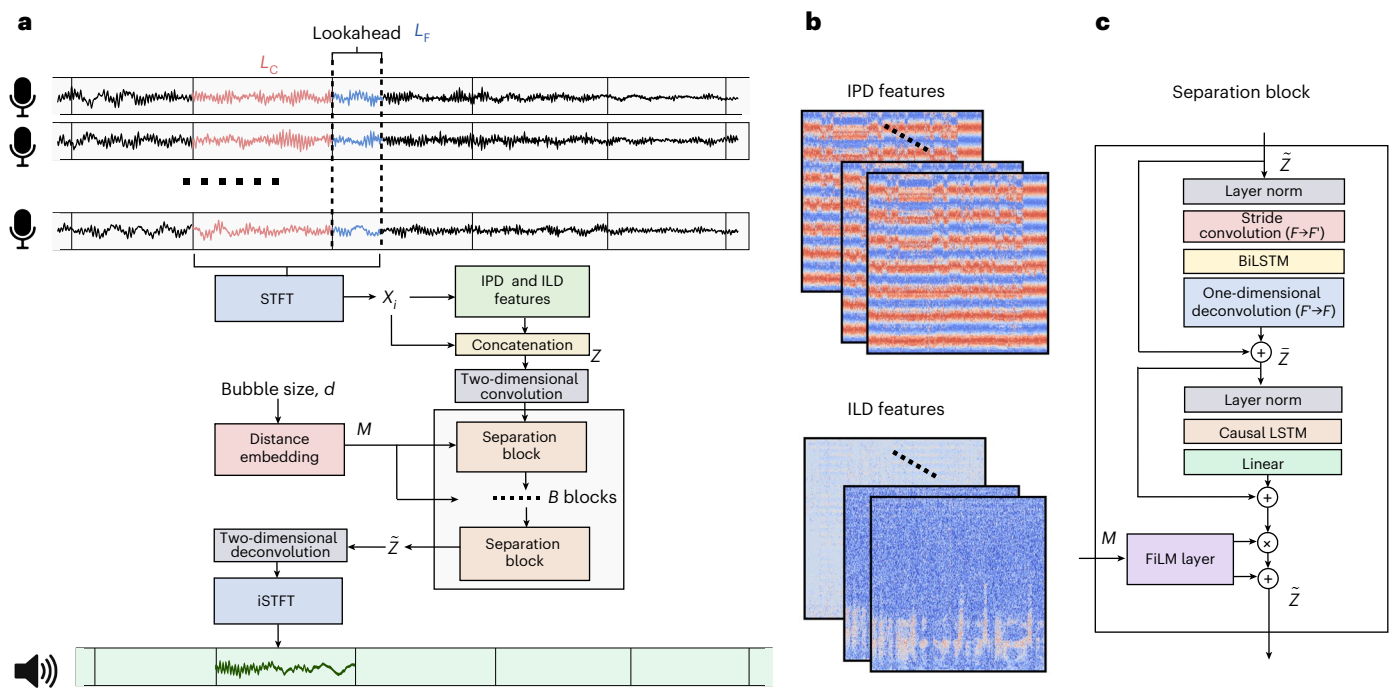


Fig. 2 | Real-time and on-device neural network architecture. **a**, We first apply STFT on L_C (ms) of the current chunk with L_F (ms) of lookahead on audio from multichannel microphones. Then, we extract the IPD and ILD features and input them into the separation module conditioned by the programmable bubble size. Then, an iSTFT converts the extracted spectrogram back to time-domain audio

and puts it into the playback buffer. **b**, Visualization of the IPD/ILD features.

c, Structure for each of our separation blocks. It applies frequency compression before the frequency-dimension LSTM to reduce complexity. After a time-domain LSTM, a FiLM layer is applied to mask the spectrogram based on distance embedding.

positions on the headset. Finally, we build an end-to-end prototype that integrates a noise-cancelling headset with a six-microphone array (Fig. 1b,c), and runs our real-time acoustic bubble network on a Raspberry Pi. We show that the headsets can suppress speakers outside the bubble (Fig. 1d).

Our neural network supports mobility, with speakers entering the bubble (Supplementary Video 1), and works with multiple overlapping speakers (Supplementary Video 2) as well as no speaker within the bubble (Supplementary Video 3). It also operates effectively in noisy and crowded environments, such as restaurants (Supplementary Video 4). Furthermore, we integrate our design with off-the-shelf noise-cancelling headsets to create real-time sound bubbles (Supplementary Video 5).

Real-time on-device neural networks for sound bubbles

Our goal is to develop real-time on-device neural networks that can run on hearable devices to create sound bubbles. To do this, we should meet four key requirements: process incoming audio in real time and on the device on an embedded CPU, and achieve an end-to-end latency of less than 20–30 ms between the audio and visual scenes; adapt to diverse reverberant conditions and wearers; support configurable bubble sizes; and work when the farther speakers are louder than those within the bubble, as well as when there are multiple or no speakers inside or outside the bubble.

Previous work on hearables achieves real-time speech enhancement for telephony⁷, but lacks distance perception and does not meet our latency needs. Semantic and directional hearing systems^{8–11} focus on extracting target sounds by semantics or angle, and not distance. Smart speaker systems that use microphone arrays^{12,13} have been proposed to achieve speech zones but fail to meet the latency constraints nor support different HRTFs inherent to wearables. Neural networks^{14,15} have been demonstrated to achieve better source separation than

statistical signal processing¹⁶. Previous work¹⁷ has proposed neural networks using a single microphone for distance-based speech extraction, but they have not shown generalization to real-world environments, are not real time or on the device and have limited performance in simulations. Other networks^{18–20} use two microphones to estimate the distance of a single sound source but are not designed for sound mixtures or source separation.

Our real-time network architecture consists of four modules (Fig. 2a): a feature encoder, a distance-embedding network, a separation module and a feature decoder. To achieve low algorithmic latency, we process the multichannel input audio in small chunks of L_C (ms) each, with lookahead L_F (ms). Our feature encoder converts the multichannel frame into a time–frequency (TF) representation using a short-time Fourier transform (STFT). In our implementation, we set L_C and L_F to 8 ms and 4 ms, respectively. We use domain-specific features, that is, interchannel phase difference (IPD) and interchannel level difference (ILD), and combine them with TF representations of the raw mixtures (Fig. 2b).

The separation module—the most computationally intensive component—then extracts the TF representations of the speakers within the bubble. We use TF-GridNet²¹, the state-of-the-art speech separation network, as our backbone separation module. However, TF-GridNet cannot run in real time on our embedded CPU (for example, Raspberry Pi). We profiled this network and observed that the frequency-dimension long short-term memory (LSTM) and attention blocks had the highest computational complexity. Thus, in our speech separation module, we remove the attention block, and we first utilize a strided convolution layer to compress the frequency dimension before applying the intraframe spectral LSTM. This reduces the sequence length used by the LSTM and therefore reduces its execution time. Then, we apply unidirectional LSTM along the time dimension (Fig. 2c). The size of the bubble is encoded as a distance-embedding mask, generated from a one-hot vector representing discrete bubble sizes.

This mask is then applied to the output of the frequency-dimension LSTM using a FiLM layer²². Finally, the TF representations of the extracted speakers are passed to an STFT decoder (that is, an inverse STFT (iSTFT) operation) to recover the time-domain signal containing only the speakers within the bubble.

For each input chunk, we also reduce the inference runtime per chunk by caching computations made as prior chunks are processed. Specifically, we maintain a set of state buffers containing intermediate computation results, which we pass to our model during inference. For example, when computing the final output of the model, the STFT decoder requires 4 ms of past context from the previous inference, which we can reuse from the appropriate state buffer. We maintain several such state buffers, including one for the initial convolution layer, a hidden-state buffer and a cell-state buffer for the intraframe LSTM in each separation block, one for the final deconvolution layer and one for the iSTFT layer. Finally, we convert our model into Open Neural Network Exchange (ONNX) for inference and run it with ONNX Runtime. We also perform ONNX-specific optimizations to fine-tune the performance for real-time operation on the embedded CPU (Methods).

We considered a six-channel microphone array similar in size to a noise-cancelling headset or smart glasses (Extended Data Fig. 1). We generated 30k rooms with random sizes ranging from $5 \times 4 \times 2 \text{ m}^3$ to $8 \times 8 \times 4 \text{ m}^3$. The reverberation of each room was randomly selected, with the wall absorption coefficient uniformly sampled from 0.1 to 0.9, and the maximum reflection order uniformly sampled between 10 and 72. In each room, 0–2 speakers were placed inside and 1–2 outside the sound bubble, with a 50% chance of adding diffuse noise. The speech dataset was split into training, validation and testing sets. The signal-to-noise ratio (SNR) of speech inside the bubble ranged from –5 dB to 5 dB, with a median of 0 dB, and in 50% of the samples, the outside interference was louder than the speech inside.

Figure 3a shows the energy decay as a speaker moves from inside to outside a 1.5 m sound bubble under various reverberant conditions, demonstrating the model's robust distance perception. Figure 3b depicts the corresponding energy decay for different sizes of sound bubbles, indicating that the bubble size can be utilized to control the distances beyond which sound sources will be attenuated. Figure 3c shows that across different reverberant conditions, the network could suppress the outside speakers with an average energy decay of 69 dB, 66 dB and 65 dB for the 1 m, 1.5 m and 2 m bubbles, respectively.

Figure 3d–f plots the output scale-invariant signal-to-distortion ratio (SI-SDR) of all the samples as a function of the input SI-SDR for 1 m, 1.5 m and 2 m bubbles, respectively. Across the three bubble sizes, approximately 97.8% of the samples showed a positive improvement. This holds true even when the input SI-SDR is less than zero, where speakers outside have higher power than those inside the bubble. With a single speaker inside the bubble, the network achieved SI-SDR improvements (SI-SDR_i) of 12.35 dB, 11.52 dB and 9.85 dB for different bubble sizes. With two speakers inside the bubble, the network achieved SI-SDR_i values of 9.02 dB, 8.55 dB and 6.94 dB, respectively.

Figure 3g and Extended Data Tables 1 and 2 compare the network configurations, showing that using IPD and ILD features improves the performance by 1.12 dB without noticeably increasing the computational complexity. Extended Data Table 3 compares different STFT parameters. Figure 3h shows that reducing the number of microphones to 4 and 2 decreases the SI-SDR_i to 11.07 dB and 9.58 dB, respectively, for a single target speaker within a 1.5 m bubble. Smart glasses improve the SI-SDR_i to 12.29 dB due to their three-dimensional microphone array configuration, which introduces more spatial diversity. Figure 3i shows that the angular and Euclidean distance difference between a speaker inside the bubble and an interfering speaker outside affects the separation performance.

Rather than using the three fixed bubble sizes, we also evaluated the feasibility of using continuous bubble sizes (Extended Data Fig. 2). To do this, instead of using one-hot vectors, we used a single

continuous value to represent the bubble size. Then, a linear layer and a rectified linear unit layer are used to convert the single value into a vector. We uniformly sample the bubble size from 0.8 m to 2 m. Then, we placed 0–2 speakers inside the bubble with 1–2 interfering speakers outside as well as background noise for training. During evaluation, we selected six different bubble sizes of 1 m, 1.2 m, 1.4 m, 1.6 m, 1.8 m and 2 m, and computed the decay–distance curves for each bubble (Extended Data Fig. 2c).

Robotic platform for real-world training data collection

Although the above experiments verify the validity of our approach and are helpful in the analysis, the models trained only on synthetic data do not generalize to real-world headsets for two key reasons: (1) audio signals will reflect off the wearer's head, causing the signal to undergo various transformations known as HRTFs that our simulations do not account for; and (2) since the headset is not perfectly rigid, the microphone positions may change slightly with different wearer's head sizes.

To ensure our neural network generalizes to unseen real-world wearers and speakers, we need a large dataset collected with real-world headsets. Existing HRTF datasets^{23,24} lack distance information, and although some prior efforts have led to the creation of distance-based HRTF datasets^{25–28}, they were used in controlled anechoic chambers. For better generalization, we need several hours of real-world data, but collecting this with human users is challenging and time-consuming due to the substantial human involvement required.

Instead, we created a robotic system that automatically collects audio recordings using a mannequin head (Fig. 4a–c). The mannequin head was mounted on a rotating robotic platform that allowed us to record audio with the head rotated at different angles. We placed a loudspeaker on an upright linear actuator to change its height (Supplementary Video 6). Figure 4d shows that our robotic system can collect all combinations of 20 angles and 3 heights within 25 min.

Real-world generalization

In addition to the mannequin dataset, we collected human wearer data in various environments to capture the acoustic properties of real humans and environments. Wearers could move their head and/or body, allowing us to train our network using both mannequin and human data for real-world evaluation. To improve the dataset diversity and generalization, we applied four data augmentation techniques: (1) shifting each audio channel by up to four samples (or 5.7 cm) to simulate changes in the microphone position, (2) adjusting the channel amplitude independently by up to 2 dB, (3) randomly zeroing out up to 512 frequency bins to enhance robustness to frequency distortions and (4) varying the audio speed by 80–120% to increase the speaker diversity.

Real-world evaluations were performed on data recorded with only human wearers of our headset in various offices, conference rooms and classrooms (Fig. 5a–e). All of the human wearers and environments were unseen during training. We tested various scenarios with 0–2 speakers inside the bubble and with 1–2 interfering speakers outside, as well as background noise.

We investigated the performance of the Orange Pi network, across different rooms, with a 1.5 m bubble. The network achieved an average energy decay of over 49 dB when no speakers were inside the bubble (Fig. 5f). It also improved the output speech SI-SDR over the noisy baseline in these previously unseen rooms (Fig. 5g). The SI-SDR_i ranged from 9.86 dB to 12.09 dB with one speaker and from 7.57 dB to 9.82 dB with two speakers inside the bubble (Fig. 5h). Furthermore, we see a relationship between the room size and separation quality. We plot the room impulse responses for a given speaker distance and angle in two vastly different-sized rooms (Fig. 5i). In smaller rooms (room E), the impulse responses reveal high-amplitude peaks compared with the direct path peak after only a few tens of milliseconds, corresponding to early reflections from nearby walls and objects. On the other hand,

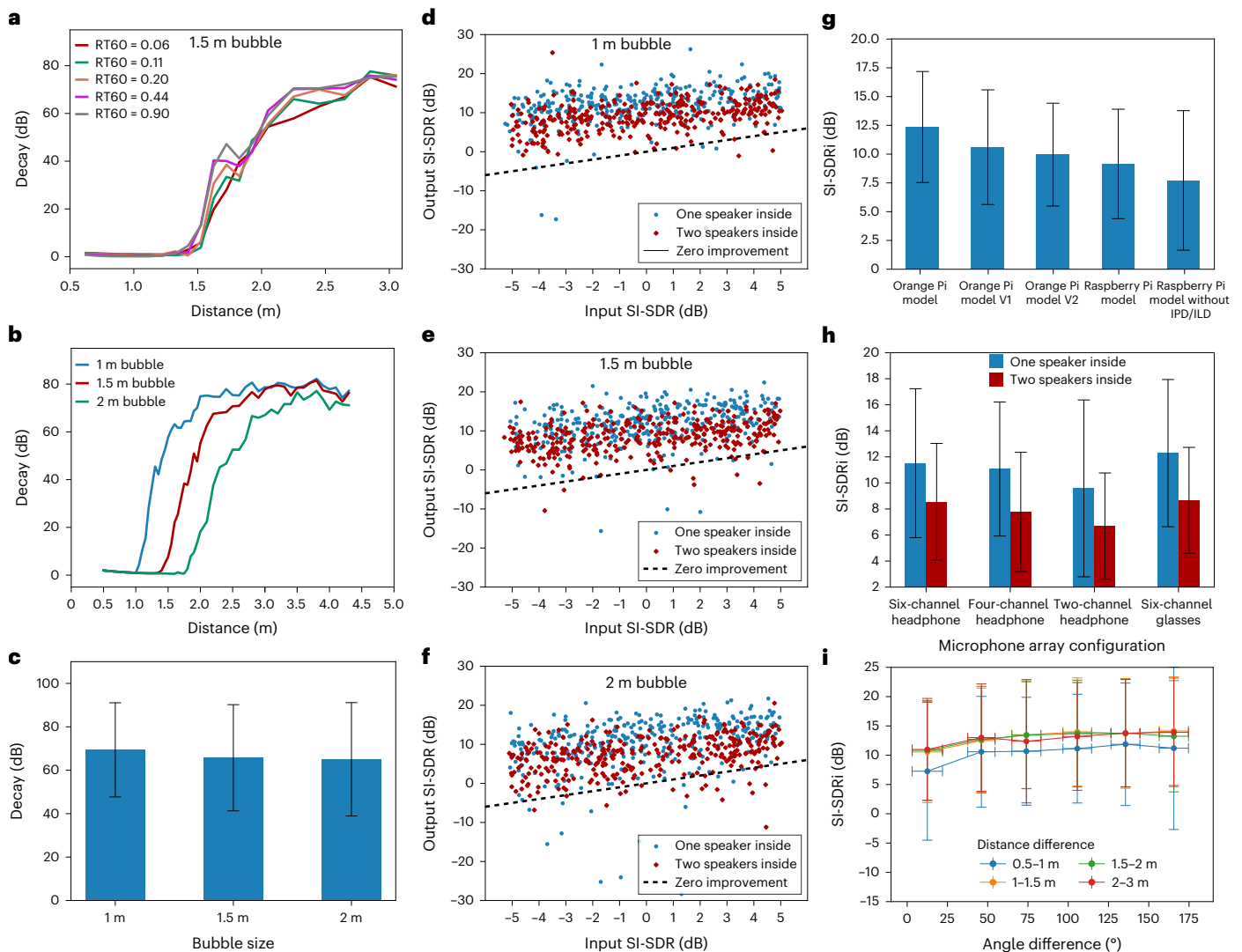


Fig. 3 | Evaluation on synthetic data. **a**, Visualization of a 1.5 m bubble under different RT60 reverberation conditions, where RT60 is the time it takes for the sound to delay by 60 dB. We plot the decay of a single speaker at different distances. **b**, Decay of a single source at different distances when we select 1 m, 1.5 m and 2 m bubbles. **c**, Decay values when no speaker is inside the three bubble sizes. The measure of the centre is the mean value and the error bar is the standard deviation ($N = 334$ per size). **d–f**, Output SI-SDR of all the samples as a function of the input SI-SDR for 1 m (**d**), 1.5 m (**e**) and 2 m (**f**) bubbles. The black dashed line is the zero-improvement line for SI-SDR. The blue points are the samples when one speaker is inside the bubble, whereas the red points are

the samples when two speakers are inside the bubble. **g**, SI-SDRi for different model configurations. The measure of the centre is the mean value and the error bar is the standard deviation ($N = 333$ per configuration). **h**, Effect of different microphone array configurations on the performance of our model. The measure of the centre is the mean value and the error bar is the standard deviation ($N = 333$ per configuration). **i**, Effect of distance and angle differences between a speaker inside the bubble and an interfering speaker outside on the separation performance. The measure of the centre is the mean value and the error bar is the standard deviation ($N = 30$ per configuration).

these peaks occur later and are lower in amplitude in larger rooms (room A).

Extended Data Table 4 shows how data augmentation improves the Raspberry Pi network's generalization to unseen mixtures, environments and users. Extended Data Table 5 compares different methods of using real-world data from mannequins and human users for training. Results show that pre-training on the mannequin dataset followed by fine-tuning on the human dataset leads to better generalization.

End-to-end integration with noise-cancelling headsets

We integrated our system with off-the-shelf noise-cancelling headsets, using a ReSpeaker six-channel microphone, Sony WH-1000XM4 headphones and a Raspberry Pi 4B. The noise-cancelling headphones suppress all sounds, allowing the system to reintroduce target sounds

from inside the bubble. When activated, the system captures audio in 8 ms chunks, processes it on the Raspberry Pi using our neural network and plays it back through the headphones. Figure 6a shows that both Raspberry Pi and Orange Pi can process 8 ms chunks of audio using the corresponding networks in real time. Figure 6b plots the noise reduction with and without active noise cancellation and shows that the headphones' noise-cancelling capability can attenuate signals across all frequencies, providing a clean slate on which we can play back only the desired sounds.

We tested the system's ability to attenuate or amplify sounds based on distance by having a participant wear it as a loudspeaker, playing speech at varying distances. Microphones inside and outside the earcups recorded the audio, with the microphone on the inside capturing the sound the wearer hears, including playback and any noise-cancellation residue (Methods). Figure 6c shows a distinct

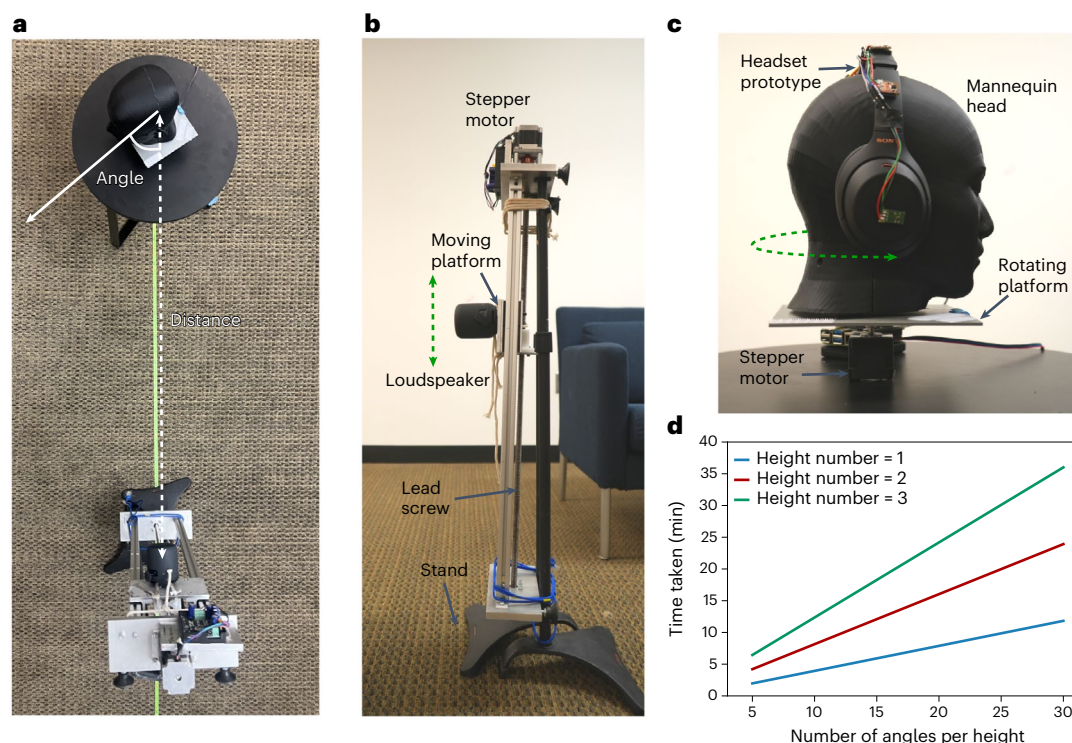


Fig. 4 | Robotic platform for real-world data collection. a, Top view of the robotic experiment setup used to collect audio recordings using a mannequin head. **b,** Linear actuator that automatically changes the height of the

loudspeaker. **c,** Rotational robotic platform on which we put a mannequin head with our designed headphones. **d,** Time taken to collect data from our automated data collection system for different numbers of angles and heights.

boundary between the speaker positions inside and outside the 1.5 m sound bubble. The signal amplification—measured as the power ratio between the sound captured inside the earcup and the outer reference microphone—drops sharply when the speaker moves outside the bubble. For example, Fig. 6d shows a speech signal in ambient noise (for example, heating, ventilation and air conditioning) as recorded by the outer microphone. When the speaker is outside the bubble, the speech is attenuated by the neural network, resulting in a faint signal inside the earcups (Fig. 6e,f). However, when the speaker is inside the bubble, the speech is preserved, denoised and clearly heard in the earcups (Fig. 6g–i).

We evaluated the perceived size of the sound bubbles by asking four participants to create a 1.5 m bubble as another person spoke at various distances. The speaker moved in 10 cm steps, and participants—with eyes closed—signalled when they perceived a sharp drop in the speaker’s voice. This was repeated five times per participant. The resulting histogram of the ‘perceptual boundary’ shows that participants generally perceived the boundary between 1.5 m and 1.6 m (Extended Data Fig. 3).

Conclusions

We have reported a hearable system capable of preserving the sounds originating inside a user-programmable bubble around the wearer and suppressing all other sounds. The system uses a neural network to identify and extract the speech from all the speakers inside and was trained on over 12 h of data collected in a variety of real-world conditions, allowing it to generalize to unseen users and environments.

Our current prototype does have some limitations. It relies on noise-cancelling systems that may allow some residual sounds. Future improvements could include using an inner microphone to monitor and mask this noise. The system runs on the device to minimize latency, which limits model complexity and performance. Although we applied architectural and runtime optimizations, further techniques, like

integer quantization, could reduce the network size and improve the inference time, though this may degrade performance.

To reduce model runtime, the source code could be optimized for target platforms using ONNX Runtime Execution Providers. These enable execution on dedicated hardware like graphical processing units, neural processing units or optimized CPU kernels. For example, Execution Providers could allow the Raspberry Pi 4B to utilize Arm NEON acceleration, whereas the Orange Pi 5B could run parts of the network on its Rockchip neural processing unit.

Our work focused on indoor environments like offices, conference rooms and noisy restaurants. Outdoor data collection is challenging due to difficulty in obtaining clean audio for training. This could be addressed by using precise acoustic simulators or collecting real-world data in quieter outdoor settings or semi-enclosed areas, such as stadiums and sports fields, with controlled noise.

As the bubble size increases, the transition between inside and outside becomes less sharp, making it harder to discern distant speakers. For instance, differences in IPD across microphones become less sensitive as the speaker moves further, reducing feature reliability for the network. Future improvements could include adding more training samples near the bubble boundary or increasing the network size. These approaches would help the network better handle distant speaker displacements. We tested bubbles up to 2 m, achieving reasonable source separation, which is sufficient for our conversation scenarios.

Our prototype meets the 20–30 ms delay requirements for augmented audio applications. Sound bubbles could also be beneficial for hearing aid applications, which have more stringent delay requirements. Power consumption is a key factor, with the system drawing 1.2 A at 5 V, giving a battery life of around 4 h when connected to a 5 V, 5,000 mAh battery. This includes non-optimized components like the microphone array. Power could be reduced by developing an application-specific integrated circuit.

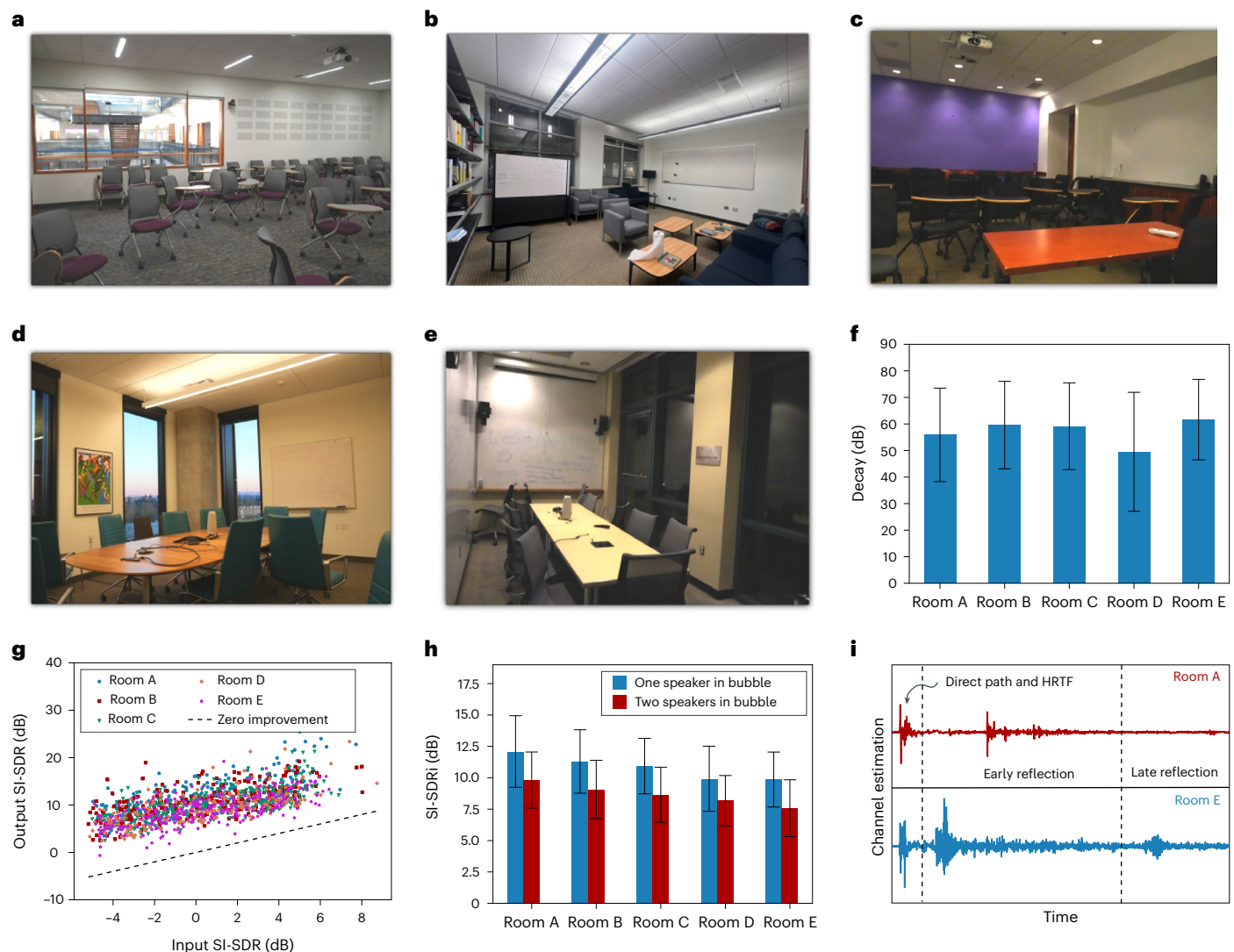


Fig. 5 | Real-world evaluation. a–e, Five different unseen test environments. The room dimensions are as follows (in m²). Room A, 11 × 9 (**a**); room B, 8 × 6 (**b**); room C, 8 × 6 (**c**); room D, 6 × 5 (**d**); room E, 6 × 3 (**e**). **f**, Mean energy decay—across 400 mixtures per room—of the incoming speech signal across different environments when there are no speakers inside a 1.5 m bubble. The error bars show the standard deviation. **g**, Output SI-SDR of all the samples in our testing

set as a function of their input SI-SDR. The samples are coloured based on their corresponding environment. **h**, Mean speech extraction quality—across 400 mixtures per room—for one and two target speakers across all the environments in our test set. The error bars show the standard deviation. **i**, Visualization of the room impulse responses for rooms A and E, showing that the smaller room has larger and denser early reflections.

Recently, companies like Google²⁹ and Phonak³⁰ have integrated AI accelerators into earbuds and hearing aids, respectively. Although their specifications are not public, commercial AI accelerators from companies like Syntiant run models with 6 million parameters at over 30 giga operations per second, consuming just tens of milliwatts³¹. In contrast, our Raspberry Pi model is much smaller, requiring about 2.35 giga multiply–add operations per second (or roughly 4.7 giga operations per second).

Methods

Our research complied with the ethical regulations of the University of Washington Institutional Review Board. Informed consent was obtained from participants. We affirm that human research participants provided informed consent for the publication of the images shown in Fig. 1.

Problem formulation

Say we have an acoustic environment with N speakers at two-dimensional locations $\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_{N-1}$. The headset is centred at

location $\mathbf{q}$, with M microphones. We denote the speech signal for speaker j received by microphone i as $s_{ij}(t)$. $s_{ij}(t)$ contains the channel response from source j to microphone i , including the multi-path effect and HRTF. We denote the background noise at microphone i by $n_i(t)$. Then, the mixture $x_i(t)$ at each microphone can be written as

$$x_i(t) = \sum_{j=0}^{N-1} s_{ij}(t) + n_i(t). \quad (1)$$

Assuming that the reference microphone index is c and the speech bubble size is d , our model $\mathcal{F}$ has to extract all the speakers within the bubble at the reference microphone c :

$$\mathcal{F}_c(x_0(t), x_1(t), \dots, x_{N-1}(t)|d) = \sum_{j=0}^{N-1} s_{cj}(t) \mathbb{I}(\|\mathbf{p}_j - \mathbf{q}\|_2 < d), \quad (2)$$

where $\mathbb{I}(\cdot)$ is an indicator function that outputs 1 if the input expression is true, or 0 otherwise. $\|\cdot\|_2$ is the Euclidean norm operation.

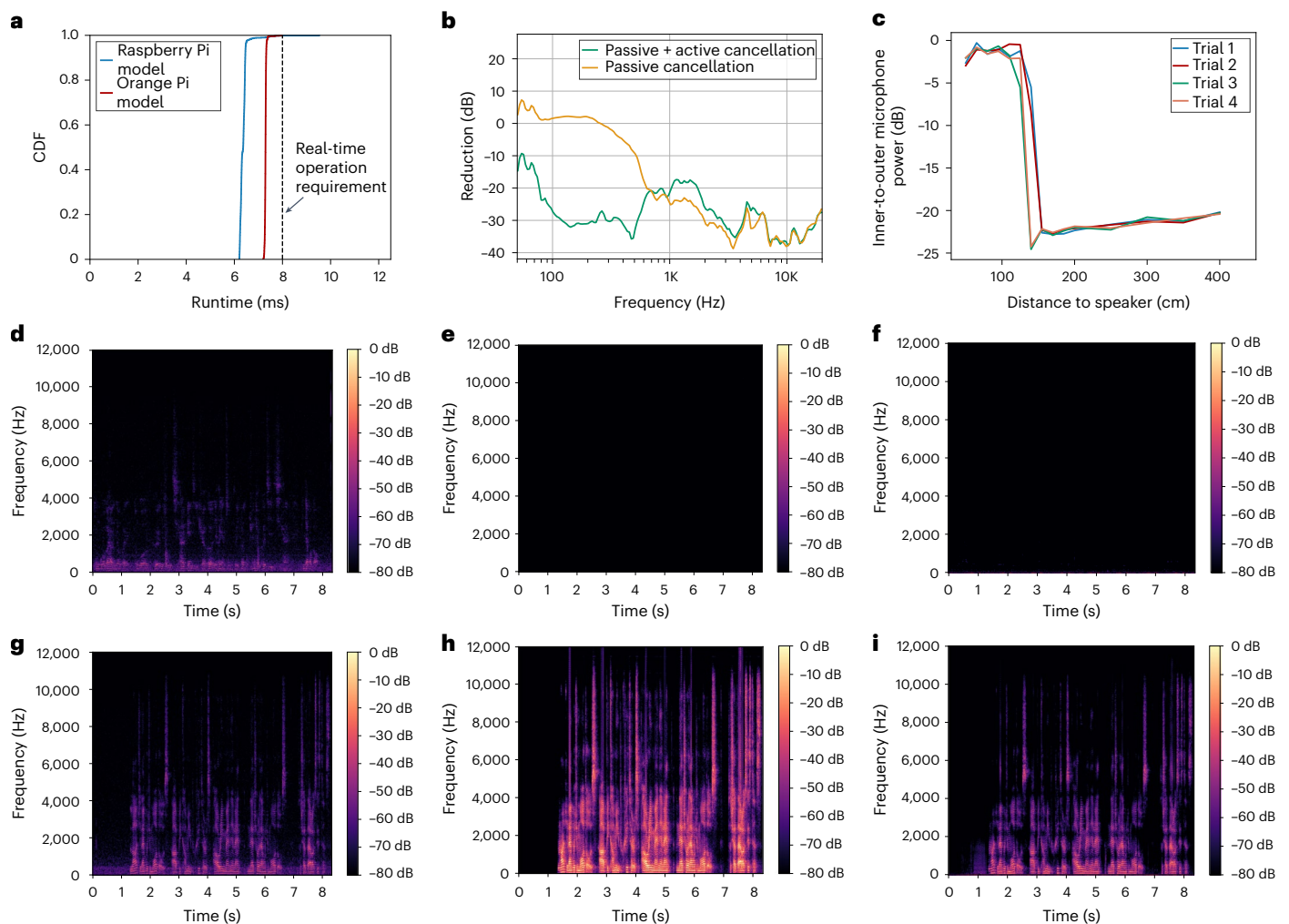


Fig. 6 | End-to-end real-time integration with noise-cancelling headsets. a, Cumulative distribution function (CDF) of the inference time of our Orange Pi and Raspberry Pi models on their respective platforms. **b**, Noise reduction achieved by the Sony WH-1000XM4 noise-cancelling headphones with and without noise cancelling turned on. **c**, Ratio of the power between the inner and outer microphones with a 1.5 m sound bubble turned on, as a function of

the speaker distance to the user. **d, e**, Spectrogram representation (**d**) of an audio signal from a speaker outside a 1.5 m bubble, as captured by an exterior microphone, which is processed by our network and played back (**e**). **f**, Signal received by a microphone inside the earcups. **g–i**, Same data as **d** (**g**), **e** (**h**) and **f** (**i**), but with only the speaker inside the 1.5 m bubble.

All the signals in our implementation are sampled at 24 kHz for two reasons: LibriTTS, our largest speech dataset, uses 24 kHz, and higher rates like 48 kHz increase runtime latency. Although a lower rate of 16 kHz could reduce latency, we chose 24 kHz to preserve more high-frequency components, aligning with modern hearing aids, which use 20–30 kHz (ref. 32). This balances latency and audio quality for our speech bubble system.

Encoder and decoder

Instead of using a learnable time-domain encoder^{15,33}, we use an STFT encoder since it provides a stable and large-enough encoder window size. This is essential to stabilize the performance under reverberation³⁴. The input audio frame is $L_c + L_f = 12$ ms long, where $L_c = 8$ ms is the current chunk size and $L_f = 4$ ms is the lookahead (Fig. 2a). Accordingly, our STFT encoder uses a discrete Fourier transform length of 288 samples, a step size of 192 samples and a Hann window. This allows us to compute the TF representation $X_i(\omega) \in \mathbb{C}^{F \times K}$ for the signal at microphone i : $x_i(t) \in \mathbb{R}^T$, where F is the number of frequency bins, K is number of time frames and T is the time duration of the input signal.

We use an iSTFT decoder with the same discrete Fourier transform length, step size and window as the encoder. This produces a

12 ms output, but we remove the last 4 ms, which will be affected by future chunks, leaving an 8 ms audio output. The 4 ms lookahead chunk is stored for overlap-and-add operation with the next chunk. Both STFT and iSTFT are implemented with convolution layers using Asteroid³⁵ and Hann windows, offering flexibility in choosing discrete Fourier transform points, unlike fast Fourier transform, which requires powers of 2.

Interchannel features

We extract the interchannel features IPD and ILD (Fig. 2b) as

$$\text{IPD}(X_i(\omega), X_j(\omega)) = \angle X_i(\omega) - \angle X_j(\omega), \quad (3)$$

$$\text{ILD}(X_i(\omega), X_j(\omega)) = 20 \log [|X_i(\omega)| / |X_j(\omega)|], \quad (4)$$

where $|\cdot|$ denotes the complex magnitude. We compute the IPD and ILD between the reference channel c and all other microphone channels and append these features to $X_{\text{feature}} \in \mathbb{R}^{(3M-3) \times F \times K}$:

$$X_{\text{feature}} = \{\cos(\text{IPD}(X_c, X_i)), \sin(\text{IPD}(X_c, X_i)), \text{ILD}(X_c, X_i)\}, \quad \forall i \in [0, M) \cap i \neq c \quad (5)$$

We stack the domain-specific features with the real and imaginary parts of the TF representations $\{\text{Real}(X_i), \text{Imag}(X_i)\}, \forall i \in [0, M)$, to obtain $Z \in \mathbb{R}^{(5M-3) \times F \times K}$. Finally, a two-dimensional convolution layer is used to associate the information across different microphone channels and features and to output a new TF representation, $\tilde{Z} \in \mathbb{R}^{E \times F \times K}$, where E is the feature channel number.

Real-time separation network

Our separation network is composed of B repeated blocks. In each individual block (Fig. 2c), a strided convolution layer $\mathcal{C} : \mathbb{R}^{E \times F \times K} \rightarrow \mathbb{R}^{E \times F' \times K}$ is applied on the frequency dimension to compress the frequency number from F to F' to reduce the computational complexity. Next, a frequency-domain bidirectional LSTM is applied to associate the information across different frequency bands followed by a transposed convolution layer to uncompress to the original frequency dimension, $\tilde{Z} \in \mathbb{R}^{E \times F \times K}$. Then, a unidirectional LSTM is applied to the frame dimension of the TF representations, $\tilde{Z}$. The hidden size for LSTM is H , which is set to 128.

Distance embedding for a programmable bubble

A distance-embedded mask $M \in \mathbb{R}^{G \times F}$, conditioned by the bubble size, is used to extract the TF representations of the speakers within the bubble from $\tilde{Z}$ using a FiLM layer. Here, G is the intermediate feature number, which is set to 4 in our implementation. The distance-embedded mask is generated from one-hot vector $d \in \mathbb{R}^D$ using a linear layer, $\mathcal{L} : \mathbb{R}^D \rightarrow \mathbb{R}^{G \times F}$, where D is the number of discrete bubble sizes. The FiLM layer is composed of two convolution layers, $\mathcal{C}_\omega : \mathbb{R}^{G \times F} \rightarrow \mathbb{R}^{E \times F}$ and $\mathcal{C}_b : \mathbb{R}^{G \times F} \rightarrow \mathbb{R}^{E \times F}$. We update $\tilde{Z}$ to extract the TF representations of the speakers within the bubble as

$$\begin{aligned}\omega_M &= \mathcal{C}_\omega(M), \\ b_M &= \mathcal{C}_b(M), \\ \tilde{Z} &= \omega_M \cdot \tilde{Z} + b_M.\end{aligned}\quad (6)$$

This updated $\tilde{Z}$ is input to the next separation block, and this process is repeated B times.

Optimizations for inference time

Our models were written and trained in PyTorch but ran inference on target hardware using ONNX Runtime. We converted the PyTorch model to ONNX (opset 19) and used ONNX Runtime v. 16.1.0 for inference. To reduce latency, we optimized the PyTorch model by (1) replacing the convolution layers with a kernel size of 1 with faster PyTorch linear layers, (2) rewriting layer normalization to fuse into a single ONNX LayerNormalization node and (3) fusing shape-related operations to minimize the ONNX Transpose nodes.

Creating a real-world distance-based audio dataset

Using our robotic platform, we played audio samples from LibriTTS³⁶ and VCTK³⁷ through a loudspeaker mounted on a linear actuator at various heights. The speaker was positioned at fixed distances from the mannequin head, and audio was recorded over 60 head orientations and three speaker heights. After testing all the angle and height combinations, the speaker was moved to a new distance. This process was repeated in four acoustic environments, with 8–11 distances per environment. Data were collected from two mannequin heads, totalling 10.8 h, all exclusively for training with minimal human intervention.

Although mannequin heads approximate human heads, they cannot fully capture human acoustic properties. Thus, we collected an additional dataset from ten human wearers in 18 acoustic environments. The dataset was split for training, validation and testing, with each environment and wearer in only one split. Training data came from two users in ten environments, validation from three users in three environments and testing from five users in five environments.

The testing environments are shown in Fig. 5a–e, with reverberation details listed in Supplementary Table 1. Training and validation environments are shown in Supplementary Fig. 2.

To collect data, users sat on a rotating chair wearing our headset, with a loudspeaker on a linear actuator placed at various distances. The loudspeaker's height was adjusted, and users rotated to random angles before audio playback. In addition to speech samples, the loudspeaker played WHAM! noise samples when outside the 1.5 m bubble. Participants were asked to naturally move their heads or bodies during playback for better generalization. This setup required more human effort than the mannequin setup, resulting in 3.4 h for training, 1.03 h for testing and 0.62 h for validation.

Dataset generation

We generated the synthetic and real-world mixtures using three open-source datasets. For speech utterances of targeted speakers and interfering speakers, we used the VCTK³⁷ and LibriTTS³⁶ datasets, whereas we used the WHAM! dataset³⁸ for the background noise samples. All of these datasets were split into three non-overlapping subsets for training, validation and testing.

We generated the synthetic dataset using the PyRoomAcoustics³⁹ simulator. Random three-dimensional rooms were created with dimensions sampled from 5 m to 8 m (length), 4 m to 8 m (width) and 2 m to 4 m (height). Wall absorption coefficients ranged from 0.1 to 0.9, with the maximum order between 10 and 72 for reverberation. A six-microphone array was randomly placed in the room at a height of 1.2–1.8 m. For each bubble size, 0–2 speakers were placed inside and 1–2 outside the bubble, with diffuse background noise added 50% of the time. Speaker distances and angles were randomly sampled, and heights varied by ± 0.4 m from the wearer's height. The input SNR uniformly ranged from -10 dB to 5 dB for training and -5 dB to 5 dB for testing and validation. The mixture of the reverberated signals of the speakers inside the bubble at the reference microphone was used as the ground-truth signal. Each sample was 5 s long, with 30k training samples, 3k validation samples and 3k testing samples.

We created real-world audio mixtures from recordings with mannequins and human wearers by combining 5 s clips from multiple speakers and noise sources in the same room. Each mixture included 0–2 speakers within 1.5 m, 1–2 speakers beyond 1.5 m and 0–1 noise sources beyond 1.5 m. To avoid stacking ambient noise, we denoised the recordings using a 500 ms noise profile for each^{40–43}, mixing one original noisy clip with the denoised clips. To vary speaker amplitude outside the bubble, we randomly scaled their signals up to two times. The ground-truth signal was the sum of denoised speakers within 1.5 m, and interfering signals were rescaled to achieve an SNR between -10 dB and 5 dB for training and -5 dB to 5 dB for testing. We generated two datasets: 60k mixtures from mannequin recordings (training only) and 20k training, 2k validation and 2k testing mixtures from human wearer recordings. The mannequin dataset included real-world ambient sounds like heating, ventilation and air conditioning noise.

Loss functions

The models were trained with two different loss functions. The first was a loss function based on the negative SNR, defined as follows:

$$\begin{aligned}L_{\text{SNR}} &= \ell_\lambda(\hat{\mathbf{s}}, \mathbf{s}) \\ &= \begin{cases} \lambda \|\hat{\mathbf{s}} - \mathbf{s}\|_1 & \text{if there are no speakers inside the bubble} \\ -10 \log \left[\frac{\|\mathbf{s}\|_2^2}{\|\hat{\mathbf{s}} - \mathbf{s}\|_2^2} \right] & \text{otherwise} \end{cases}\end{aligned}\quad (7)$$

Here $\mathbf{s}$ is the target signal, $\hat{\mathbf{s}}$ is the network output signal, $\|\cdot\|_1$ is the L1-norm (equivalently, the sum of element-wise absolute differences) and $\lambda = 50$ is a weighting factor.

We also used a combination of L1-loss and a variant of the multiresolution spectrogram loss, defined as follows:

$$L_{\text{MultiRes}} = \ell_{R,k_1}(\hat{\mathbf{s}}, \mathbf{s}) = k_1 \|\hat{\mathbf{s}} - \mathbf{s}\|_1 + \frac{1}{|R|} \sum_{R_i \in R} \left| \|\text{STFT}_{R_i}(\hat{\mathbf{s}})\| - \|\text{STFT}_{R_i}(\mathbf{s})\| \right|_1. \quad (8)$$

Here R is a set of STFT resolutions, $k_1 = 20$ is an L1-weighting factor and STFT_{R_i} is the STFT operation with resolution R_i . We used three resolutions, where we vary the fast Fourier transform length over {1,024, 2,048, 512}, the step size over {120, 240, 50} and the window length over {600, 1,200, 240}. We use L_{SNR} in the pre-training stage and L_{MultiRes} in the fine-tuning stage. Further details are provided in the next section.

Training procedure

To train the model for the synthetic dataset, we trained a single model on the 30k dataset, containing 1 m, 1.5 m and 2 m bubbles, as well as 0–2 speakers inside the bubbles. We first trained the model using SNR-based loss L_{SNR} for 150 epochs, and then we fine-tuned the model on the original 30k dataset using multiresolution loss L_{MultiRes} for 100 epochs. The initial learning rate was set to 2×10^{-3} . For all the models and training runs, we used the Adam optimizer, a batch size of 8 and a gradient clipping of 1.

We trained the real-world model in two stages. First, we pre-trained the networks on the mannequin dataset (150 epochs for Raspberry Pi and 161 for Orange Pi) using data augmentations. The learning rate was increased from 2×10^{-4} to 2×10^{-3} over 10 epochs, held constant for 20 epochs and then reduced by 0.95 every 2 epochs. Next, we fine-tuned the models on the human wearer dataset for 200 epochs, halving the learning rate if validation loss did not decrease for 8 epochs. Pre-trained weights were used for fine-tuning with the L_{MultiRes} loss function. Loss curves are shown in Supplementary Fig. 4.

The models in Extended Data Table 4 were trained only on the human wearer dataset using L_{SNR} as a loss function. Each data augmentation technique was applied with some probability in the order presented in the table, from left to right. When we only use one data augmentation technique, this augmentation was applied with 50% probability. When applying multiple augmentations together, each was applied with 30% probability.

We trained the Raspberry Pi model in three ways (Extended Data Table 5): (1) using only mannequin data, (2) using only human wearer data and (3) pre-training on mannequin data followed by fine-tuning with human data. All the strategies used L_{SNR} as the loss function.

Evaluation procedure

All testing was performed with the model parameters that yield the lowest validation loss.

In Fig. 3a,b, we placed the microphone array randomly in the room and positioned a single speaker at various distances. For each distance, we played a random speech sample from the test set, processed it with the Orange Pi model and repeated this 30 times to compute the average decay, defined as the reduction in output signal energy relative to the input:

$$\text{Decay} = 10 \log \left[\frac{\|x_c\|_2^2}{\|\hat{s}\|_2^2} \right], \quad (9)$$

where x_c is the input mixture of reference microphone c and $\hat{s}$ is the output of neural network.

To generate Fig. 3a, we initialized a room ($8 \times 8 \times 3$ m) and varied the reverberation parameters to compute the decay curves for different RT60 values. For Fig. 3b, we used the same room with a fixed RT60 (0.11 s) to compute the decay–distance curves for different bubble sizes (1 m, 1.5 m and 2 m). In Fig. 3c, we computed decay for three bubble sizes with no speaker inside. Figure 3d–f shows the scatter plots of

input and output SI-SDRs for one or two speakers inside the bubbles. In Fig. 3g, we trained five model configurations and computed the SI-SDR for one speaker inside a 1 m bubble. Differences in configurations and metrics are detailed in Extended Data Tables 1 and 2.

In Fig. 3h, we trained three additional Orange Pi models with different microphone array configurations: four-channel and two-channel arrays on headphones, and a six-channel array on glasses. In Fig. 3i, we created a testing set with one target and one interfering speaker, consisting of 720 data points. The absolute angle difference between speakers was uniformly sampled across six intervals: [0, 30), [30, 60), [60, 90), [90, 120), [120, 150) or [150, 180). The distance difference between the two speakers is uniformly sampled from four intervals: [0.5 m, 1 m), [1 m, 1.5 m), [1.5 m, 2 m) or [2 m, 3 m). We explore other factors in Supplementary Fig. 3.

End-to-end integration with noise-cancelling headsets

We built the hardware by desoldering microphones from the array and resoldering four onto individual printed circuit boards (Supplementary Fig. 5a). The remaining two microphones came from off-the-shelf evaluation boards (AMM-2742-T-EB-R) (Supplementary Fig. 5b). All the microphones were wired to the array driver, which connected to the Raspberry Pi's general purpose input/output pins. The microphone circuits were mounted on the headphone band and earcups, with audio played back via an auxiliary cable connected to the driver's output jack.

To create sound bubbles, we used a JACK server to transfer 48 kHz audio in 128-sample chunks. A JACK client processed these samples, downsampling them to 24 kHz using zita-resampler. After receiving 8 ms of samples, they were passed to ONNX Runtime for network processing. The model's output was amplified by 10 dB, upsampled and sent to the headset via the JACK client. The 10 dB amplification ensures that the output is clear and audible, applied equally inside and outside the bubble.

To evaluate the system's ability to suppress sounds outside the bubble, a participant wore the system with microphones inside the earcups. A loudspeaker played 5 s speech clips at various distances, and we recorded audio from the inner and outer microphones. A 0.2–15 kHz chirp synchronized the recordings, using the 6–10 kHz range to mark the start. This experiment was repeated four times across 15 distances, with a 15 cm resolution below 2 m and 50 cm beyond 2 m.

Reporting summary

Further information on research design is available in the Nature Portfolio Reporting Summary linked to this article.

Data availability

The LibriTTS dataset is available at <https://www.openslr.org/60> and the WHAM! dataset is available at <http://wham.whisper.ai/>. All the synthetic datasets are available via Dryad at <https://doi.org/10.5061/dryad.r7sqv9smv> (ref. 44). The real-world datasets are available from the corresponding authors upon request. Source data are provided with this paper.

Code availability

The source code is available via GitHub at https://github.com/chentuocho/Sound_Bubble.git.

References

- Blauert, J. *Spatial Hearing: The Psychophysics of Human Sound Localization* (MIT Press, 1997).
- Zahorik, P., Brungart, D. S. & Bronkhorst, A. W. Auditory distance perception in humans: a summary of past and present research. *Acta Acust. United Acust.* **91**, 409–420 (2005).
- Bradley, J., Reich, R. D. & Norcross, S. On the combined effects of signal-to-noise ratio and room acoustics on speech intelligibility. *J. Acoust. Soc. Am.* **106**, 1820–1828 (1999).

4. Bronkhorst, A. W. The cocktail party phenomenon: a review of research on speech intelligibility in multiple-talker conditions. *Acta Acust. United Acust.* **86**, 117–128 (2000).
5. Stone, M. A. & Moore, B. C. Tolerable hearing aid delays. I. Estimation of limits imposed by the auditory path alone using simulated hearing losses. *Ear Hear.* **20**, 182–192 (1999).
6. Gupta, R., Ranjan, R., He, J., Gan, W.-S. & Peksi, S. Acoustic transparency in hearables for augmented reality audio: hear-through techniques review and challenges. In *Audio Engineering Society Conference on Audio for Virtual and Augmented Reality* (Audio Engineering Society, 2020).
7. Chatterjee, I. et al. ClearBuds: wireless binaural earbuds for learning-based speech enhancement. In *Proc. 20th Annual International Conference on Mobile Systems, Applications and Services* 384–396 (Association for Computing Machinery, 2022).
8. Yang, S., Wisdom, S., Gnegy, C., Lyon, R. F. & Savla, S. Listening with Googlears: low-latency neural multiframe beamforming and equalization for hearing aids. In *INTERSPEECH* 3939–3943 (International Speech Communication Association, 2022).
9. Veluri, B., Itani, M., Chan, J., Yoshioka, T. & Gollakota, S. Semantic hearing: programming acoustic scenes with binaural hearables. In *Proc. 36th Annual ACM Symposium on User Interface Software and Technology* 89 (Association for Computing Machinery, 2023).
10. Wang, A., Kim, M., Zhang, H. & Gollakota, S. Hybrid neural networks for on-device directional hearing. *Proc. AAAI Conf. Artif. Intell.* **36**, 11421–11430 (2022).
11. Veluri, B., Itani, M., Chen, T., Yoshioka, T. & Gollakota, S. Look once to hear: target speech hearing with noisy examples. In *Proc. CHI Conference on Human Factors in Computing Systems* 37 (Association for Computing Machinery, 2024).
12. Itani, M., Chen, T., Yoshioka, T. & Gollakota, S. Creating speech zones with self-distributing acoustic swarms. *Nat. Commun.* **14**, 5684 (2023).
13. Gu, R. & Luo, Y. Rezero: region-customizable sound extraction. *IEEE/ACM Trans. Audio Speech Lang. Process.* **32**, 2576–2589 (2024).
14. Luo, Y., Chen, Z., Mesgarani, N. & Yoshioka, T. End-to-end microphone permutation and number invariant multi-channel speech separation. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* 6394–6398 (IEEE, 2020).
15. Luo, Y. & Mesgarani, N. Conv-TasNet: surpassing ideal time–frequency magnitude masking for speech separation. *IEEE/ACM Trans. Audio Speech Lang. Process.* **27**, 1256–1266 (2019).
16. Zheng, Y., Goubran, R. & El-Tanany, M. Robust near-field adaptive beamforming with distance discrimination. *IEEE Speech Audio Process.* **12**, 478–488 (2004).
17. Patterson, K., Wilson, K., Wisdom, S. & Hershey, J. R. Distance-based sound separation. In *INTERSPEECH* 901–905 (International Speech Communication Association, 2022).
18. Ding, J., Ke, Y., Cheng, L., Zheng, C. & Li, X. Joint estimation of binaural distance and azimuth by exploiting deep neural networks. *J. Acoust. Soc. Am.* **147**, 2625–2635 (2020).
19. Kushwaha, S. S., Roman, I. R., Fuentes, M. & Bello, J. P. Sound source distance estimation in diverse and dynamic acoustic conditions. In *IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)* 1–5 (IEEE, 2023).
20. Georganti, E., May, T., van de Par, S. & Mourjopoulos, J. Extracting sound–source–distance information from binaural signals. In *The Technology of Binaural Listening* 171–199 (Springer, 2013).
21. Wang, Z.-Q. et al. TF-GridNet: integrating full- and sub-band modeling for speech separation. *IEEE/ACM Trans. Audio Speech Lang. Process.* **31**, 3221–3236 (2023).
22. Perez, E., Strub, F., de Vries, H., Dumoulin, V. & Courville, A. FiLM: visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence* (PKP Publishing Services Network, 2018).
23. Hummersone, C. Binaural impulse responses captured in real rooms. *GitHub repository* (2016). Available at: <https://github.com/loSR-Surrey/RealRoomBRIIRs.git>
24. Satongar, D., Lam, Y. W. & Pike, C. Measurement and analysis of a spatially sampled binaural room impulse response dataset. *Proc. 21st Int. Congr. Sound Vib.*, Beijing, China, 13–17 (2014).
25. Qu, T. et al. Distance-dependent head-related transfer functions measured with high spatial resolution using a spark gap. *IEEE Trans. Audio, Speech, Language Process.* **17**, 1124–1132 (2009).
26. Wierstorf, H., Geier, M. & Spors, S. A free database of head-related impulse response measurements in the horizontal plane with multiple distances. In *130th Convention of the Audio Engineering Society*, eBrief 6 (AES, 2011).
27. Xie, B. *Head-Related Transfer Function and Virtual Auditory Display* (J. Ross Publishing, 2013).
28. Salvador, C. D., Sakamoto, S., Treviño, J. & Suzuki, Y. Dataset of near-distance head-related transfer functions calculated using the boundary element method. In *AES International Conference on Spatial Reproduction—Aesthetics and Science* 41–49 (Audio Engineering Society, 2018).
29. Park, B. Meet Pixel Buds Pro 2, the first Buds built for Gemini. *Google Blog* <https://blog.google/products/pixel/google-pixel-buds-pro-2/> (2024).
30. Phonak. Audéo™ Sphere hearing aids. *Phonak*. Available at: <https://www.phonak.com/en-us/hearing-devices/hearing-aids/audéo-sphere> (2024).
31. Medici, G. Syntiant unveils NDP250 Neural Decision Processor with Next-Gen Core 3 architecture. *Syntiant* <https://www.syntiant.com/news/syntiant-unveils-ndp250-neural-decision-processor-with-next-gen-core-3-architecture> (2024).
32. Hoppe, U. & Hesse, G. Hearing aids: indications, technology, adaptation and quality control. *GMS Curr. Top Otorhinolaryngol. Head Neck Surg.* **16**, Doc08 (2017).
33. Thakker, M., Eskimez, S. E., Yoshioka, T., & Wang, H. Fast real-time personalized speech enhancement: end-to-end enhancement network (E3Net) and knowledge distillation. In *INTERSPEECH* 991–995 (International Speech Communication Association, 2022).
34. Cord-Landwehr, T. et al. Monaural source separation: from anechoic to reverberant environments. In *2022 International Workshop on Acoustic Signal Enhancement (IWAENC)* 1–5 (IEEE, 2022).
35. Pariente, M. et al. Asteroid: the PyTorch-based audio source separation toolkit for researchers. In *INTERSPEECH* 2637–2641 (International Speech Communication Association, 2020).
36. Zen, H. et al. LibriTTS: a corpus derived from LibriSpeech for text-to-speech. In *INTERSPEECH* 1526–1530 (International Speech Communication Association, 2019).
37. Veaux, C. et al. *Superseded-CSTR VCTK Corpus: English Multi-speaker Corpus for CSTR Voice Cloning Toolkit* (Centre for Speech Technology Research, Univ. Edinburgh, 2016).
38. Wichern, G. et al. WHAM!: extending speech separation to noisy environments. *INTERSPEECH* 1368–1372 (International Speech Communication Association, 2019).
39. Scheibler, R., Bezzam, E., & Dokmanić, I., PyRoomAcoustics: a Python package for audio room simulation and array processing algorithms. In *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* 351–355 (IEEE, 2018).
40. Sainburg, T., Thielk, M. & Gentner, T. Q. Finding, visualizing and quantifying latent structure across diverse animal vocal repertoires. *PLoS Comput. Biol.* **16**, e1008228 (2020).
41. Brown, J. C. Calculation of a constant Q spectral transform. *J. Acoust. Soc. Am.* **89**, 425–434 (1991).

42. Veluri, B. et al. Real-time target sound extraction. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* 1–5 (IEEE, 2023).
43. Hu, Y. et al. DCCRN: deep complex convolution recurrent network for phase-aware speech enhancement. In *INTERSPEECH* 2472–2476 (International Speech Communication Association, 2020).
44. Chen, T. et al. Hearable devices with sound bubbles. *Dryad* <https://doi.org/10.5061/dryad.r7sqv9smv> (2024).

Acknowledgements

We thank B. Veluri, A. Raghavan and M. Kim for their feedback on the project. T.C., M.I. and S.G. are funded by Moore Inventor Fellow award number 10617, the UW CoMotion award and the National Science Foundation.

Author contributions

T.C. and M.I. designed and evaluated the neural networks with guidance from S.E.E., T.Y. and S.G. The real-world data collection robotic system was built by T.C. and M.I. T.C. conducted the data and signal analyses. The end-to-end system with noise-cancelling headsets was designed by M.I. All authors wrote the paper. This study was conceptualized by T.C., M.I. and S.G.

Competing interests

S.G. is a co-founder of Jeeva Wireless and Waveley Diagnostics. S.E.E. is an employee of Microsoft and T.Y. is an employee of AssemblyAI. T.C., M.I. and S.G. are inventors on a US provisional patent application 63/625,732 submitted by the University of Washington, which is related to this work.

Additional information

Extended data is available for this paper at <https://doi.org/10.1038/s41928-024-01276-z>.

Supplementary information The online version contains supplementary material available at <https://doi.org/10.1038/s41928-024-01276-z>.

Correspondence and requests for materials should be addressed to Tuocho Chen, Malek Itani or Shyamnath Gollakota.

Peer review information *Nature Electronics* thanks Dong Ma, José Luis Blanco-Murillo and the other, anonymous, reviewer(s) for their contribution to the peer review of this work.

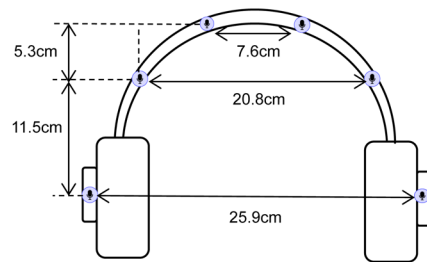
Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

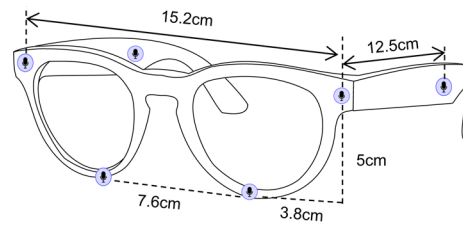
Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

© The Author(s), under exclusive licence to Springer Nature Limited 2024

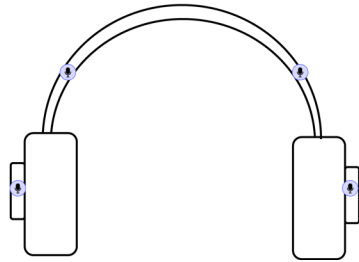
A. Layout of headphone



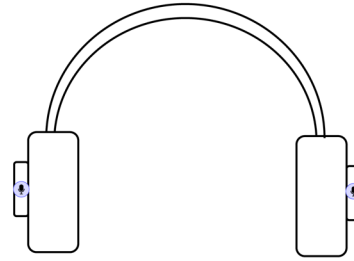
B. Layout of glasses



C. Layout of 4-mic configuration

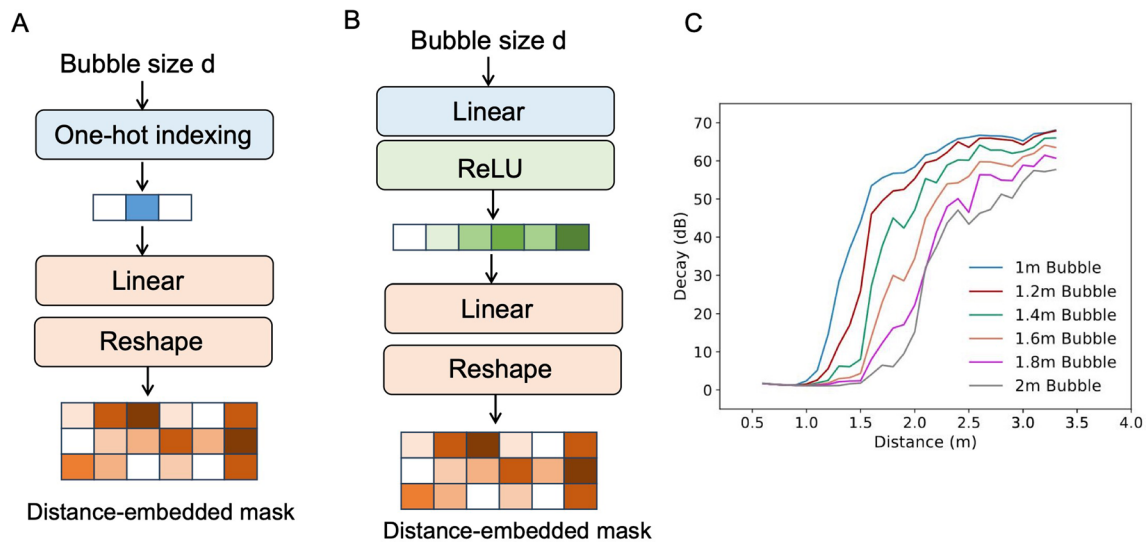


D. Layout of 2-mic configuration

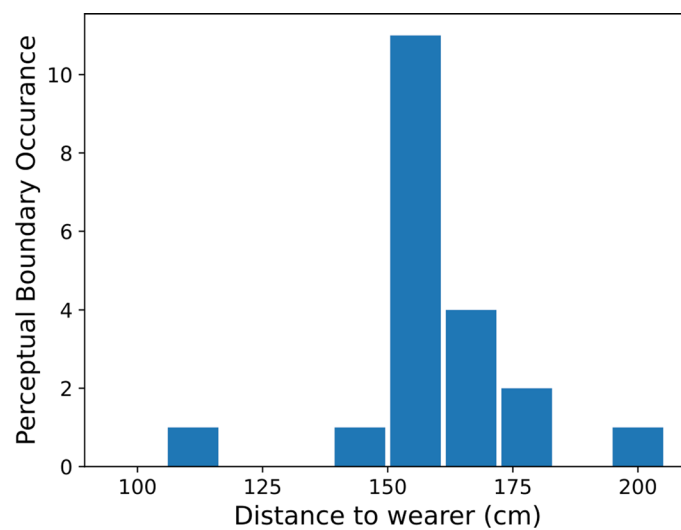


Extended Data Fig. 1 | The layout of the microphone arrays in different configurations. **A.** shows the layout of the 6-channel microphone array on the headphones used for our real-world and synthetic dataset. **B.** shows the layout 6-channel microphone array on the glasses for the synthetic evaluation.

C. shows the layout of the 4-channel microphone array on the headphones for the synthetic evaluation. **D.** shows the layout of the 2-channel microphone array on the headphones for the synthetic evaluation.



Extended Data Fig. 2 | Continuous bubble size evaluation. **A.** The distance embedding module for the discrete bubble size. **B.** The distance embedding module for the continuous bubble size. **C.** plots the decay of a single source at different distances for six different bubble sizes (1, 1.2, 1.4, 1.6, 1.8, 2m) using the continuous-bubble size model.



Extended Data Fig. 3 | Perceptual boundary evaluation. Perceptual boundary of the sound bubbles as reported by the participants using our system.

Extended Data Table 1 | Model configurations and corresponding parameters

Model Config	B	E (feature)	Compress F	IPD/ILD features	Atten	MACs	Params
TF-GridNet	6	32	-	-	✓	86.26M	652.2K
OrangePi Model	6	32	-	✓	-	73.76M	501.4K
OrangePi Model V1	6	32	✓	✓	-	51.7M	599.9 K
OrangePi Model V2	3	32	✓	✓	-	26.44M	304.89 K
RaspberryPi Model	3	16	✓	✓	-	18.77 M	226.28 K
RaspberryPi Model w/o IPD/ILD	3	16	✓	-	-	18.46 M	224.12 K

Extended Data Table 2 | Model configuration evaluation

Model Config	Decay (0spk)	SISDRi (1spk)	PESQ (1spk)	STOI (1spk)	SISDRi (2spk)
TF-GridNet	62.70	12.69	3.048	0.929	9.09
OrangePi Model	69.39	12.35	2.989	0.929	9.02
OrangePi Model V1	61.27	10.6	2.668	0.905	7.48
OrangePi Model V2	55.54	9.95	2.559	0.901	6.9
RaspberryPi Model	55.12	9.14	2.371	0.882	6.2
RaspberryPi Model w/o IPD/ILD	50.05	7.71	2.122	0.799	5.08

Extended Data Table 3 | Effect of different STFT parameters

$L_C + L_F$	Decay (0spk)	SISDRi (1spk)	SISDRi (2spk)	MACs (per sec)	Params
8ms + 4ms	69.39	12.35	9.02	9.22G	501.40K
10ms + 2ms	68.35	12.03	8.66	7.38G	501.40K
12ms + 0ms	58.24	9.44	6.95	6.15G	501.40K
6ms + 3ms	67.76	12.40	8.71	9.24G	500.97K
4ms + 2ms	61.2	11.95	8.34	9.29G	500.53K

Extended Data Table 4 | Performance comparison of the Raspberry Pi network trained with different data augmentation techniques

Sample Shift	Speed	Frequency Masking	Channel Gain	Single Target SI-SDRi (dB)
-	-	-	-	7.19
✓	-	-	-	7.72
-	✓	-	-	7.88
-	-	✓	-	7.08
-	-	-	✓	7.34
✓	✓	-	✓	8.15
✓	✓	✓	✓	8.29

Extended Data Table 5 | Performance comparison of the Raspberry Pi network trained by combining multiple real world datasets

Training strategy	Single Target SI-SDRi (dB)
Mannequin data only	7.67
Human data only	8.29
Pre-train with mannequin data, fine-tune with human data	9.06

Reporting Summary

Nature Portfolio wishes to improve the reproducibility of the work that we publish. This form provides structure for consistency and transparency in reporting. For further information on Nature Portfolio policies, see our [Editorial Policies](#) and the [Editorial Policy Checklist](#).

Statistics

For all statistical analyses, confirm that the following items are present in the figure legend, table legend, main text, or Methods section.

n/a	Confirmed
☐	☒ The exact sample size (<i>n</i>) for each experimental group/condition, given as a discrete number and unit of measurement
☒	☐ A statement on whether measurements were taken from distinct samples or whether the same sample was measured repeatedly
☒	☐ The statistical test(s) used AND whether they are one- or two-sided <i>Only common tests should be described solely by name; describe more complex techniques in the Methods section.</i>
☒	☐ A description of all covariates tested
☒	☐ A description of any assumptions or corrections, such as tests of normality and adjustment for multiple comparisons
☐	☒ A full description of the statistical parameters including central tendency (e.g. means) or other basic estimates (e.g. regression coefficient) AND variation (e.g. standard deviation) or associated estimates of uncertainty (e.g. confidence intervals)
☒	☐ For null hypothesis testing, the test statistic (e.g. <i>F</i> , <i>t</i> , <i>r</i>) with confidence intervals, effect sizes, degrees of freedom and <i>P</i> value noted <i>Give P values as exact values whenever suitable.</i>
☒	☐ For Bayesian analysis, information on the choice of priors and Markov chain Monte Carlo settings
☒	☐ For hierarchical and complex designs, identification of the appropriate level for tests and full reporting of outcomes
☒	☐ Estimates of effect sizes (e.g. Cohen's <i>d</i> , Pearson's <i>r</i>), indicating how they were calculated

Our web collection on [statistics for biologists](#) contains articles on many of the points above.

Software and code

Policy information about [availability of computer code](#)

Data collection	A custom app based on Python3.9 and ONNXRuntime 1.16.2 was used to record data on the headset. A custom app based on Python3.9 was used to control the rotation of the robotic platform.
Data analysis	We used Python3.9 script to analyze results and plot the graphs in the paper. Code to reproduce these results are available in Github repository: https://github.com/chentuochoao/Sound_Bubble

For manuscripts utilizing custom algorithms or software that are central to the research but not yet described in published literature, software must be made available to editors and reviewers. We strongly encourage code deposition in a community repository (e.g. GitHub). See the Nature Portfolio [guidelines for submitting code & software](#) for further information.

Data

Policy information about [availability of data](#)

All manuscripts must include a [data availability statement](#). This statement should provide the following information, where applicable:

- Accession codes, unique identifiers, or web links for publicly available datasets
- A description of any restrictions on data availability
- For clinical datasets or third party data, please ensure that the statement adheres to our [policy](#)

The LibriTTS dataset we used is available at <https://www.openslr.org/60> and the WHAM! dataset we used is available at <http://wham.whisper.ai/>. We provided all synthetic dataset in <https://doi.org/10.5061/dryad.r7sqv9smv>. The real-world dataset is available upon request.

Research involving human participants, their data, or biological material

Policy information about studies with [human participants or human data](#). See also policy information about [sex, gender \(identity/presentation\), and sexual orientation](#) and [race, ethnicity and racism](#).

Reporting on sex and gender	We had 2 males for training and 3 males for validation set, and 2 males and 3 females for testing set.
Reporting on race, ethnicity, or other socially relevant groupings	1 white, 3 Indians, and 6 Asians.
Population characteristics	For age, 2 participants are between 30-40 years old, 8 participants are between 20-30 years old.
Recruitment	All participants in our study were adults. The recruit process is that we send the recruitment notice in public channel in University of Washington communication channel.
Ethics oversight	Our study included human participants to evaluate the system performance with human wearer. This experiment was approved by the University of Washington Institutional Review Board: STUDY00018895. We also add this statement to "Method" Section in our manuscript.

Note that full information on the approval of the study protocol must also be provided in the manuscript.

Field-specific reporting

Please select the one below that is the best fit for your research. If you are not sure, read the appropriate sections before making your selection.

☐ Life sciences ☒ Behavioural & social sciences ☐ Ecological, evolutionary & environmental sciences

For a reference copy of the document with all sections, see [nature.com/documents/nr-reporting-summary-flat.pdf](https://www.nature.com/documents/nr-reporting-summary-flat.pdf)

Life sciences study design

All studies must disclose on these points even when the disclosure is negative.

Sample size	Describe how sample size was determined, detailing any statistical methods used to predetermine sample size OR if no sample-size calculation was performed, describe how sample sizes were chosen and provide a rationale for why these sample sizes are sufficient.
Data exclusions	Describe any data exclusions. If no data were excluded from the analyses, state so OR if data were excluded, describe the exclusions and the rationale behind them, indicating whether exclusion criteria were pre-established.
Replication	Describe the measures taken to verify the reproducibility of the experimental findings. If all attempts at replication were successful, confirm this OR if there are any findings that were not replicated or cannot be reproduced, note this and describe why.
Randomization	Describe how samples/organisms/participants were allocated into experimental groups. If allocation was not random, describe how covariates were controlled OR if this is not relevant to your study, explain why.
Blinding	Describe whether the investigators were blinded to group allocation during data collection and/or analysis. If blinding was not possible, describe why OR explain why blinding was not relevant to your study.

Behavioural & social sciences study design

All studies must disclose on these points even when the disclosure is negative.

Study description	We collected the audio wave file which is recorded from the headset when the participants were wearing it. All data and analysis are quantitative.
Research sample	We recruited undergraduates, graduate student and faculty at University of Washington. The ages range from 20-40 years old.
Sampling strategy	To collect the real-human data, we randomly selected the participant and acoustic environment for each collection. Training data came from 2 users in 10 environments, validation from 3 users in 3 environments, and testing from 5 users in 5 environments. There is no wearer and environment overlap across training, validation and testing. Totally, our real-human data have 3.4 hours for training, 1.03 hours for testing, and 0.62 hours for validation. In addition, we have collected 10.8 hour with our Robotic Platform for training.
Data collection	To collect data, users sat on a rotating chair wearing our headset, with a loudspeaker on a linear actuator placed at various distances. The loudspeaker's height was adjusted, and users rotated to random angles before audio playback. Participants were asked to naturally move their heads or bodies during playback for better generalization.

Timing	August 2023 to November 2023.
Data exclusions	No data excluded.
Non-participation	No dropout and decline.
Randomization	Participants were randomly assigned to training, validation and testing set.

Ecological, evolutionary & environmental sciences study design

All studies must disclose on these points even when the disclosure is negative.

Study description	<i>Briefly describe the study. For quantitative data include treatment factors and interactions, design structure (e.g. factorial, nested, hierarchical), nature and number of experimental units and replicates.</i>
Research sample	<i>Describe the research sample (e.g. a group of tagged <i>Passer domesticus</i>, all <i>Stenocereus thurberi</i> within Organ Pipe Cactus National Monument), and provide a rationale for the sample choice. When relevant, describe the organism taxa, source, sex, age range and any manipulations. State what population the sample is meant to represent when applicable. For studies involving existing datasets, describe the data and its source.</i>
Sampling strategy	<i>Note the sampling procedure. Describe the statistical methods that were used to predetermine sample size OR if no sample-size calculation was performed, describe how sample sizes were chosen and provide a rationale for why these sample sizes are sufficient.</i>
Data collection	<i>Describe the data collection procedure, including who recorded the data and how.</i>
Timing and spatial scale	<i>Indicate the start and stop dates of data collection, noting the frequency and periodicity of sampling and providing a rationale for these choices. If there is a gap between collection periods, state the dates for each sample cohort. Specify the spatial scale from which the data are taken</i>
Data exclusions	<i>If no data were excluded from the analyses, state so OR if data were excluded, describe the exclusions and the rationale behind them, indicating whether exclusion criteria were pre-established.</i>
Reproducibility	<i>Describe the measures taken to verify the reproducibility of experimental findings. For each experiment, note whether any attempts to repeat the experiment failed OR state that all attempts to repeat the experiment were successful.</i>
Randomization	<i>Describe how samples/organisms/participants were allocated into groups. If allocation was not random, describe how covariates were controlled. If this is not relevant to your study, explain why.</i>
Blinding	<i>Describe the extent of blinding used during data acquisition and analysis. If blinding was not possible, describe why OR explain why blinding was not relevant to your study.</i>
Did the study involve field work?	☐ Yes ☐ No

Field work, collection and transport

Field conditions	<i>Describe the study conditions for field work, providing relevant parameters (e.g. temperature, rainfall).</i>
Location	<i>State the location of the sampling or experiment, providing relevant parameters (e.g. latitude and longitude, elevation, water depth).</i>
Access & import/export	<i>Describe the efforts you have made to access habitats and to collect and import/export your samples in a responsible manner and in compliance with local, national and international laws, noting any permits that were obtained (give the name of the issuing authority, the date of issue, and any identifying information).</i>
Disturbance	<i>Describe any disturbance caused by the study and how it was minimized.</i>

Reporting for specific materials, systems and methods

We require information from authors about some types of materials, experimental systems and methods used in many studies. Here, indicate whether each material, system or method listed is relevant to your study. If you are not sure if a list item applies to your research, read the appropriate section before selecting a response.

Materials & experimental systems

n/a	Involved in the study
☒	☐ Antibodies
☒	☐ Eukaryotic cell lines
☒	☐ Palaeontology and archaeology
☒	☐ Animals and other organisms
☒	☐ Clinical data
☒	☐ Dual use research of concern
☒	☐ Plants

Methods

n/a	Involved in the study
☒	☐ ChIP-seq
☒	☐ Flow cytometry
☒	☐ MRI-based neuroimaging

Antibodies

Antibodies used	Describe all antibodies used in the study; as applicable, provide supplier name, catalog number, clone name, and lot number.
Validation	Describe the validation of each primary antibody for the species and application, noting any validation statements on the manufacturer's website, relevant citations, antibody profiles in online databases, or data provided in the manuscript.

Eukaryotic cell lines

Policy information about [cell lines and Sex and Gender in Research](#)

Cell line source(s)	State the source of each cell line used and the sex of all primary cell lines and cells derived from human participants or vertebrate models.
Authentication	Describe the authentication procedures for each cell line used OR declare that none of the cell lines used were authenticated.
Mycoplasma contamination	Confirm that all cell lines tested negative for mycoplasma contamination OR describe the results of the testing for mycoplasma contamination OR declare that the cell lines were not tested for mycoplasma contamination.
Commonly misidentified lines (See ICLAC register)	Name any commonly misidentified cell lines used in the study and provide a rationale for their use.

Palaeontology and Archaeology

Specimen provenance	Provide provenance information for specimens and describe permits that were obtained for the work (including the name of the issuing authority, the date of issue, and any identifying information). Permits should encompass collection and, where applicable, export.
Specimen deposition	Indicate where the specimens have been deposited to permit free access by other researchers.
Dating methods	If new dates are provided, describe how they were obtained (e.g. collection, storage, sample pretreatment and measurement), where they were obtained (i.e. lab name), the calibration program and the protocol for quality assurance OR state that no new dates are provided.
☐ Tick this box to confirm that the raw and calibrated dates are available in the paper or in Supplementary Information.	
Ethics oversight	Identify the organization(s) that approved or provided guidance on the study protocol, OR state that no ethical approval or guidance was required and explain why not.

Note that full information on the approval of the study protocol must also be provided in the manuscript.

Animals and other research organisms

Policy information about [studies involving animals; ARRIVE guidelines](#) recommended for reporting animal research, and [Sex and Gender in Research](#)

Laboratory animals	For laboratory animals, report species, strain and age OR state that the study did not involve laboratory animals.
Wild animals	Provide details on animals observed in or captured in the field; report species and age where possible. Describe how animals were caught and transported and what happened to captive animals after the study (if killed, explain why and describe method; if released, say where and when) OR state that the study did not involve wild animals.
Reporting on sex	Indicate if findings apply to only one sex; describe whether sex was considered in study design, methods used for assigning sex. Provide data disaggregated for sex where this information has been collected in the source data as appropriate; provide overall

numbers in this Reporting Summary. Please state if this information has not been collected. Report sex-based analyses where performed, justify reasons for lack of sex-based analysis.

Field-collected samples

For laboratory work with field-collected samples, describe all relevant parameters such as housing, maintenance, temperature, photoperiod and end-of-experiment protocol OR state that the study did not involve samples collected from the field.

Ethics oversight

Identify the organization(s) that approved or provided guidance on the study protocol, OR state that no ethical approval or guidance was required and explain why not.

Note that full information on the approval of the study protocol must also be provided in the manuscript.

Clinical data

Policy information about [clinical studies](#)

All manuscripts should comply with the ICMJE [guidelines for publication of clinical research](#) and a completed [CONSORT checklist](#) must be included with all submissions.

Clinical trial registration

Provide the trial registration number from ClinicalTrials.gov or an equivalent agency.

Study protocol

Note where the full trial protocol can be accessed OR if not available, explain why.

Data collection

Describe the settings and locales of data collection, noting the time periods of recruitment and data collection.

Outcomes

Describe how you pre-defined primary and secondary outcome measures and how you assessed these measures.

Dual use research of concern

Policy information about [dual use research of concern](#)

Hazards

Could the accidental, deliberate or reckless misuse of agents or technologies generated in the work, or the application of information presented in the manuscript, pose a threat to:

No	Yes
☐	☐ Public health
☐	☐ National security
☐	☐ Crops and/or livestock
☐	☐ Ecosystems
☐	☐ Any other significant area

Experiments of concern

Does the work involve any of these experiments of concern:

No	Yes
☐	☐ Demonstrate how to render a vaccine ineffective
☐	☐ Confer resistance to therapeutically useful antibiotics or antiviral agents
☐	☐ Enhance the virulence of a pathogen or render a nonpathogen virulent
☐	☐ Increase transmissibility of a pathogen
☐	☐ Alter the host range of a pathogen
☐	☐ Enable evasion of diagnostic/detection modalities
☐	☐ Enable the weaponization of a biological agent or toxin
☐	☐ Any other potentially harmful combination of experiments and agents

Plants

Seed stocks	Report on the source of all seed stocks or other plant material used. If applicable, state the seed stock centre and catalogue number. If plant specimens were collected from the field, describe the collection location, date and sampling procedures.
Novel plant genotypes	Describe the methods by which all novel plant genotypes were produced. This includes those generated by transgenic approaches, gene editing, chemical/radiation-based mutagenesis and hybridization. For transgenic lines, describe the transformation method, the number of independent lines analyzed and the generation upon which experiments were performed. For gene-edited lines, describe the editor used, the endogenous sequence targeted for editing, the targeting guide RNA sequence (if applicable) and how the editor was applied.
Authentication	Describe any authentication procedures for each seed stock used or novel genotype generated. Describe any experiments used to assess the effect of a mutation and, where applicable, how potential secondary effects (e.g. second site T-DNA insertions, mosaicism, off-target gene editing) were examined.

ChIP-seq

Data deposition

- ☐ Confirm that both raw and final processed data have been deposited in a public database such as [GEO](#).
- ☐ Confirm that you have deposited or provided access to graph files (e.g. BED files) for the called peaks.

Data access links <i>May remain private before publication.</i>	For "Initial submission" or "Revised version" documents, provide reviewer access links. For your "Final submission" document, provide a link to the deposited data.
Files in database submission	Provide a list of all files available in the database submission.
Genome browser session (e.g. UCSC)	Provide a link to an anonymized genome browser session for "Initial submission" and "Revised version" documents only, to enable peer review. Write "no longer applicable" for "Final submission" documents.

Methodology

Replicates	Describe the experimental replicates, specifying number, type and replicate agreement.
Sequencing depth	Describe the sequencing depth for each experiment, providing the total number of reads, uniquely mapped reads, length of reads and whether they were paired- or single-end.
Antibodies	Describe the antibodies used for the ChIP-seq experiments; as applicable, provide supplier name, catalog number, clone name, and lot number.
Peak calling parameters	Specify the command line program and parameters used for read mapping and peak calling, including the ChIP, control and index files used.
Data quality	Describe the methods used to ensure data quality in full detail, including how many peaks are at FDR 5% and above 5-fold enrichment.
Software	Describe the software used to collect and analyze the ChIP-seq data. For custom code that has been deposited into a community repository, provide accession details.

Flow Cytometry

Plots

Confirm that:

- ☐ The axis labels state the marker and fluorochrome used (e.g. CD4-FITC).
- ☐ The axis scales are clearly visible. Include numbers along axes only for bottom left plot of group (a 'group' is an analysis of identical markers).
- ☐ All plots are contour plots with outliers or pseudocolor plots.
- ☐ A numerical value for number of cells or percentage (with statistics) is provided.

Methodology

Sample preparation	Describe the sample preparation, detailing the biological source of the cells and any tissue processing steps used.
Instrument	Identify the instrument used for data collection, specifying make and model number.
Software	Describe the software used to collect and analyze the flow cytometry data. For custom code that has been deposited into a community repository, provide accession details.

Cell population abundance

Describe the abundance of the relevant cell populations within post-sort fractions, providing details on the purity of the samples and how it was determined.

Gating strategy

Describe the gating strategy used for all relevant experiments, specifying the preliminary FSC/SSC gates of the starting cell population, indicating where boundaries between "positive" and "negative" staining cell populations are defined.

☐ Tick this box to confirm that a figure exemplifying the gating strategy is provided in the Supplementary Information.

Magnetic resonance imaging

Experimental design

Design type

Indicate task or resting state; event-related or block design.

Design specifications

Specify the number of blocks, trials or experimental units per session and/or subject, and specify the length of each trial or block (if trials are blocked) and interval between trials.

Behavioral performance measures

State number and/or type of variables recorded (e.g. correct button press, response time) and what statistics were used to establish that the subjects were performing the task as expected (e.g. mean, range, and/or standard deviation across subjects).

Acquisition

Imaging type(s)

Specify: functional, structural, diffusion, perfusion.

Field strength

Specify in Tesla

Sequence & imaging parameters

Specify the pulse sequence type (gradient echo, spin echo, etc.), imaging type (EPI, spiral, etc.), field of view, matrix size, slice thickness, orientation and TE/TR/flip angle.

Area of acquisition

State whether a whole brain scan was used OR define the area of acquisition, describing how the region was determined.

Diffusion MRI

☐ Used

☐ Not used

Preprocessing

Preprocessing software

Provide detail on software version and revision number and on specific parameters (model/functions, brain extraction, segmentation, smoothing kernel size, etc.).

Normalization

If data were normalized/standardized, describe the approach(es): specify linear or non-linear and define image types used for transformation OR indicate that data were not normalized and explain rationale for lack of normalization.

Normalization template

Describe the template used for normalization/transformation, specifying subject space or group standardized space (e.g. original Talairach, MNI305, ICBM152) OR indicate that the data were not normalized.

Noise and artifact removal

Describe your procedure(s) for artifact and structured noise removal, specifying motion parameters, tissue signals and physiological signals (heart rate, respiration).

Volume censoring

Define your software and/or method and criteria for volume censoring, and state the extent of such censoring.

Statistical modeling & inference

Model type and settings

Specify type (mass univariate, multivariate, RSA, predictive, etc.) and describe essential details of the model at the first and second levels (e.g. fixed, random or mixed effects; drift or auto-correlation).

Effect(s) tested

Define precise effect in terms of the task or stimulus conditions instead of psychological concepts and indicate whether ANOVA or factorial designs were used.

Specify type of analysis: ☐ Whole brain ☐ ROI-based ☐ Both

Statistic type for inference

Specify voxel-wise or cluster-wise and report all relevant parameters for cluster-wise methods.

(See [Eklund et al. 2016](#))

Correction

Describe the type of correction and how it is obtained for multiple comparisons (e.g. FWE, FDR, permutation or Monte Carlo).

Models & analysis

n/a	Involvement in the study	
☐	☐ Functional and/or effective connectivity	
☐	☐ Graph analysis	
☐	☐ Multivariate modeling or predictive analysis	
Functional and/or effective connectivity		Report the measures of dependence used and the model details (e.g. Pearson correlation, partial correlation, mutual information).
Graph analysis		Report the dependent variable and connectivity measure, specifying weighted graph or binarized graph, subject- or group-level, and the global and/or node summaries used (e.g. clustering coefficient, efficiency, etc.).
Multivariate modeling and predictive analysis		Specify independent variables, features extraction and dimension reduction, model, training and evaluation metrics.